

ICT1011: Computer Organisation and Architecture

Tutorial 4: Assembler Directives and Executable Instructions

Q1. Referencing to the Instruction Set Architecture Documents

- (a) Fill in the instruction lengths and number of execution cycles for each of the mnemonic listed in Table Q1.
- (b) Comment on the potential impact(s) that can be caused by the execution of the instruction stated in Q1.1.
- (c) Compare the following pair of instructions and comment on the similarities or differences in their length and execution cycles.
 - i. Q1.1 and Q1.2
 - ii. Q1.2 and Q1.3
 - iii. Q1.3 and Q1.4

No.	Mnemonics	Instruction length (in bytes)	No. of cycles
Q1.1	<code>mov.w R5,R1</code>		
Q1.2	<code>mov.w #10,R10</code>		
Q1.3	<code>add.w #10,R10</code>		
Q1.4	<code>add.b &2000H,R12</code>		
Q1.5	<code>call #44h</code>		

Table Q1 – Instruction length and number of execution states of several MSP430 instructions

Q2. Assembly Directives and Executable Instructions

- (a) Using an appropriate assembler directive, give the line of assembly code that would create an array to store 60 integers. The array should be given the address label **Array**.
(Note: Assume that an integer has a range of -32768 to 32767).
- (b) Using an appropriate assembler directive, give the line of assembly code that would assign a decimal value of **60,000** to a constant with the **address** label **Num1**.
- (c) With reference to the program shown in Figure Q2, answer the following questions.
 - i. Draw a **byte-sized memory map** of the contents in the .data section. Your memory map should be clearly labelled. All known contents should be indicated within the memory map. Appropriate address labels should be associated with their respective locations. You may assume that the location counter begins from the address 0.
 - ii. Assume that the .data and .text sections are allocated and linked to physical memory locations starting from 0x2400 and 0x4400 respectively, give the hexadecimal value for the all the labels declared.

```

CodeSeg      .equ      0x4400
DataSeg      .set      0x3400

              .data
Vdata        .byte     3
Wdata        .word     10,-1
Xdata        .space    2
Ydata        .string   "BA", "BY"
Zdata        .word     99h

              .text
RESET        mov.w     #10,R4
Copy         mov.w     #Vdata,R10
              mov.w     #DataSeg,R11
Loop         dec.b     R4
              jne      Loop

              .end

```

Figure Q2 – Source listing of an assembly language program